

introduccion.html

- **actualizarRelojIntro:** Creé un reloj digital en tiempo real que se activa solo al cargar la página. Obtengo la hora del sistema con `new Date()`, extraigo las horas, minutos y segundos, y aplico `.padStart(2, '0')` para que los números siempre tengan dos dígitos (como "09" en vez de "9"). Al final, el script usa `setTimeout` para reejecutarse cada segundo, pintando la hora en un contenedor HTML en un bucle infinito
- **fsaludoIntro:** Diseñé este script para mostrar un mensaje de bienvenida si el usuario escribe en el campo de texto. Con un condicional, compruebo si hay datos, si los hay, lanzo una alerta con su nombre y detalles de la web, y si el campo está vacío, se le recuerda con otra alerta que debe poner su nombre
- **fcambiarFondoIntro:** Permito cambiar el color de fondo de la web al instante escribiendo un color en inglés en el cuadro de texto. El código verifica que no esté vacío y modifica la propiedad `style.backgroundColor` borrando cualquier imagen previa con "none", si no hay texto, avisa al usuario con una alerta
- **fcambiarTextoIntro:** Con esta función modifico un párrafo de la interfaz usando el valor del cuadro de texto. Si el campo contiene datos, cambio el contenido del elemento `parrafo_dinamico_intro` mediante `textContent`, si está vacío, el código salta al bloque alternativo (`else`) para restaurar el mensaje por defecto
- **fcuentaAtrasIntro:** Programé una cuenta atrás que se muestra en un bloque específico de la página. Uso un bucle que empieza en 5 y baja hasta 1 sumando cada número a una cadena de texto, al terminar, añado un salto de línea con un mensaje final e inyecto todo el resultado usando `.innerHTML`
- **fpiramideIntro:** Genero una pirámide de "estrellas" basada en la nota que el usuario le da a la web. Primero aseguro que la entrada sea un número válido y mayor a cero usando `parseInt` e `isNaN`, y luego uso dos bucles anidados, uno controla las filas y el otro va acumulando los asteriscos (*) y saltos de línea para pintarlos en pantalla

maravillas.html

- **focultarFotosMaravillas:** Oculto las imágenes de esta sección para limpiar la pantalla. Capturo todas las etiquetas de imagen con `getElementsByTagName` y las recorro con un bucle, si el ID de la imagen es diferente a "foto_rotativa", cambio su propiedad `style.visibility` a "hidden" para esconderla sin romper el diseño
- **fverFotosMaravillas:** Es el interruptor inverso del script anterior para mostrar la galería otra vez. Escaneo todo el documento para buscar los elementos de tipo imagen y, mediante un bucle, cambio su propiedad de estilo de nuevo a "visible" para que reaparezcan al momento
- **fcambiarFotoCarrusel:** Gestiono un carrusel que cambia las imágenes de las banderas de forma cíclica. Utilizo un array con las rutas de las fotos y en un contador que sube con cada

clic, si este contador iguala o supera la longitud del array, se reinicia a cero y actualiza el atributo .src de la imagen con ID foto_rotativa

- **fcompararAlturasMaravillas:** Comparo las alturas de dos monumentos para ver cuál es más grande. Convierto el texto de los campos en números enteros usando `parseInt` y, tras validar que sean correctos, ejecuto una estructura condicional que avisa por alertas si el primero es más alto, si lo es el segundo, o si miden lo mismo

gastronomia.html

- **fanadirPlatoGastro:** Agrego nuevos platos al menú de la carta de forma dinámica. Leo el texto del campo de entrada y, si no está en blanco, uso el método `.push()` para meter ese valor al final de un array global llamado `arrayPlatosSeleccionados`, luego muestro un aviso, vacío el campo y actualizo la lista visual con otra función
- **feliminarPlatoGastro:** Quito el último plato que se haya añadido al menú. Primero reviso con la propiedad `.length` que el array tenga elementos y, si es así, uso el método `.pop()` para borrar el último registro, avisando con una alerta qué plato se eliminó antes de actualizar la pantalla
- **fvisualizarMenuGastro:** Estructuro el contenido del array para que se pueda leer ordenadamente en la página. Localizo el contenedor `resultado_lista_gastro` y uso un bucle `for` para recorrer el array, en cada vuelta voy sumando la posición (`i + 1`), el nombre del plato y un salto de línea, metiendo todo en la propiedad `.innerHTML`
- **fcalcularSumaGastro:** Calculo el coste total de una comida multiplicando el precio por el número de personas. Capturo los valores de los inputs usando el operador `|| 0` para evitar errores si están vacíos, valido que sean mayores a cero y muestro el resultado final en euros a través de una alerta

inscripcion.html

- **fentrada_parque:** Controló el acceso usando condicionales según la edad y datos del usuario. Si el campo de edad principal está vacío detengo el script, pero si tiene datos evalúo tres casos, los menores de 18 necesitan poner la edad de un acompañante mayor de edad, entre 18 y 60 entran directo, y los mayores de 60 necesitan tener marcada la casilla de autorización para poder ingresar
- **colorletra:** Es una función que cambia el color del texto de un número según su valor. Convierto el texto del elemento `numero` en un entero con `parseInt` y miro su signo: si es positivo pongo la fuente en verde ("green"), si es negativo la tiño de rojo ("red"), y si es cero la devuelvo a negro
- **fincrementar, fdecrementar y fresetear:** Esta funcion controla en conjunto un contador digital de billetes. Las funciones de sumar y restar leen el texto del contenedor, hacen la operación matemática (+1 o -1), actualizan el texto en pantalla y llaman a `colorletra()`, la función de resetear vuelve el contador a cero y restaura el color negro

- **fcalcularTablaMultiplicarBilletes:** Calculo el equipaje total que va en la bodega multiplicando los pasajes por las maletas de cada persona. Copio el valor del contador y del input de maletas, compruebo que haya pasajes activos y que no se pongan maletas negativas, hago la multiplicación y que salga el resultado detallado en un párrafo

ENLACE:

<https://github.com/LuiggiTantalean/PaginaWebScriptLuiggi.git>